

Bases de Datos I
Serie 3: Modificación de estructuras y datos

Objetivos

Los ejercicios buscan practicar:

- CREATE TABLE.
- IDENTITY.
- Tipos INT, VARCHAR y DATE.
- NULL, NOT NULL, y DEFAULT.
- Incorporación mediante ALTER TABLE de:
 - PRIMARY KEY, cuando corresponda;
 - FOREIGN KEY;
 - UNIQUE;
 - CHECK.
- INSERT, UPDATE y DELETE.
- Modificación de estructuras que **ya contienen datos**.
- Incorporación de columnas sin pérdida de información.
- Cambio de tipos de datos.
- Tratamiento correcto de valores NULL mediante IS NULL.
- Reconocimiento del efecto de restricciones ante operaciones de actualización o eliminación.
- Separación de datos clasificatorios en nuevas tablas.
- Identificación del **orden correcto de un proceso de transformación**

Ejercicio 1. Registro de clientes

Una pequeña empresa necesita registrar sus clientes. De cada cliente se conoce un número generado automáticamente por el sistema, apellido y nombre, correo electrónico y fecha de alta.

El correo electrónico puede desconocerse inicialmente, pero cuando se informa no puede repetirse entre dos clientes.

Se requiere:

1. Crear la tabla Cliente.
2. Utilizar un código numérico generado automáticamente como clave primaria.
3. Definir apellido y nombre como dato obligatorio.
4. Permitir que el correo electrónico tenga valor nulo.
5. Asignar como fecha de alta predeterminada *2026-01-01*.
6. Agregar posteriormente una restricción que impida registrar dos clientes con el mismo correo electrónico.
7. Insertar al menos cuatro clientes, incluyendo uno sin correo electrónico.
8. Intentar insertar otro cliente repitiendo un correo ya existente y analizar el resultado.

Ejercicio 2. Productos con control de precio

Un comercio posee productos identificados mediante un código generado automáticamente. De cada producto se registra **descripción, precio y estado**.

Inicialmente se decidió que el estado sea un texto de hasta 10 caracteres y que, si no se especifica, tome el valor ACTIVO.

Se requiere:

1. Crear la tabla Producto.
2. Definir adecuadamente las columnas y sus condiciones de nulabilidad.
3. Utilizar DEFAULT para el estado.
4. Insertar cinco productos.
5. Agregar posteriormente una restricción CHECK para impedir precios menores o iguales a cero.
6. Intentar registrar un producto con precio negativo.
7. Modificar mediante UPDATE el precio de uno de los productos existentes.
8. Analizar qué ocurre si se intenta asignar precio cero después de creada la restricción.

Ejercicio 3. Empleados y número de documento

Una empresa registra empleados mediante un código interno generado automáticamente. Además, almacena **apellido y nombre, DNI y fecha de ingreso**.

El DNI no puede repetirse.

Se requiere:

1. Crear la tabla Empleado.

2. Establecer el código de empleado como IDENTITY.
3. Definir DNI, apellido y nombre como obligatorios.
4. Incorporar una restricción UNIQUE sobre DNI mediante ALTER TABLE.
5. Insertar cuatro empleados.
6. Intentar insertar un empleado con DNI repetido.
7. Modificar mediante UPDATE el DNI de un empleado por un valor que ya pertenece a otro.
8. Explicar qué restricción interviene en cada operación rechazada.

Secuencia A: clientes y pedidos

Los ejercicios 4 a 6 pueden resolverse utilizando el mismo lote de datos.

Ejercicio 4. Incorporación de pedidos

A partir de la tabla Cliente del ejercicio 1, la empresa decide registrar pedidos.

Cada pedido posee un número generado automáticamente, una fecha, un importe y el código del cliente que lo realizó.

Se requiere:

1. Crear la tabla Pedido.
2. Definir correctamente su clave primaria.
3. Insertar inicialmente algunos pedidos correspondientes a clientes existentes.
4. Incorporar mediante ALTER TABLE una FOREIGN KEY entre Pedido y Cliente.
5. Intentar registrar un pedido utilizando un código de cliente inexistente.
6. Explicar por qué la última operación es aceptada o rechazada.

Ejercicio 5. Eliminación de un cliente con pedidos

Continuando con las tablas Cliente y Pedido, se detecta que uno de los clientes desea darse de baja.

Se requiere:

1. Identificar un cliente que tenga pedidos registrados.
2. Intentar eliminarlo con DELETE.
3. Observar y explicar el resultado considerando la restricción de integridad referencial existente.
4. Identificar posteriormente un cliente que no posea pedidos y eliminarlo.
5. Comparar ambos resultados.

El objetivo no es utilizar subconsultas ni JOIN. Los códigos necesarios pueden indicarse directamente en las sentencias.

Ejercicio 6. Pedidos pendientes de fecha de entrega

Se decide incorporar a Pedido la fecha en que cada pedido fue entregado. Como existen pedidos todavía pendientes, este dato no siempre estará disponible.

Se requiere:

1. Modificar la tabla para agregar fecha_entrega.
2. Elegir correctamente entre NULL y NOT NULL.
3. Actualizar la fecha de entrega de algunos pedidos.
4. Obtener o identificar mediante una condición sencilla los pedidos que todavía no tienen fecha de entrega, utilizando correctamente la comparación con NULL. (SELECT usando WHERE sencillo)
5. Intentar resolver la misma condición mediante = NULL.
6. Explicar la diferencia observada.

Ejercicio 7. Cambio de longitud de una descripción

La tabla Producto fue creada originalmente con la descripción definida como VARCHAR(30). Con el uso del sistema se descubre que algunas descripciones necesitan hasta 80 caracteres. La tabla ya contiene datos.

Se requiere:

1. Modificar la definición de la columna para permitir hasta 80 caracteres.
2. Comprobar que los datos existentes se mantienen.
3. Actualizar uno de los productos utilizando una descripción de más de 30 caracteres.
4. Explicar por qué esta modificación de estructura puede realizarse sin eliminar y recrear la tabla.

Ejercicio 8. Incorporación de un dato obligatorio en una tabla existente

La tabla Empleado ya contiene cinco registros. La empresa decide incorporar una columna estado, de tipo VARCHAR(10), que deberá ser obligatoria para todos los empleados. Inicialmente todos los empleados existentes deben considerarse ACTIVO.

Se requiere determinar y ejecutar un procedimiento que:

1. Agregue la nueva columna.
2. Evite perder los registros existentes.
3. Garantice que todos los empleados actuales tengan un valor válido.
4. Deje finalmente la columna como obligatoria.
5. Establezca ACTIVO como valor predeterminado para nuevos empleados.

Importante: además de las sentencias, indicar el **orden** en que deben realizarse las **operaciones** y justificarlo.

Ejercicio 9. Control del estado de un pedido

En la tabla Pedido se agrega una columna denominada estado, que solamente debe admitir los valores PENDIENTE, ENVIADO y ENTREGADO.

Se requiere:

1. Incorporar la columna.
2. Asignar PENDIENTE a los registros existentes.
3. Definir el valor PENDIENTE como predeterminado para nuevos pedidos.
4. Crear mediante ALTER TABLE una restricción CHECK que admita solamente los tres valores definidos.
5. Realizar un UPDATE que cambie un pedido a ENVIADO.
6. Intentar modificar otro pedido asignándole CANCELADO.
7. Explicar el resultado.

Secuencia B: asignación de empleados a sectores

Ejercicio 10. Sectores de una empresa

Una empresa posee sectores identificados por código y nombre. Cada empleado puede estar asignado a un sector.

Se parte de la tabla Empleado ya utilizada anteriormente.

Se requiere:

1. Crear una tabla Sector con código numérico y nombre.
2. Impedir que existan dos sectores con el mismo nombre.
3. Insertar los sectores Administración, Ventas y Depósito.
4. Agregar a Empleado una columna codigo_sector.
5. Asignar mediante UPDATE un sector a cada empleado.
6. Crear posteriormente la restricción FOREIGN KEY.
7. Intentar asignar a un empleado el código de un sector inexistente.

Ejercicio 11. Empleados temporalmente sin sector

La empresa desea permitir que un empleado quede temporalmente sin sector asignado.

Utilizando las tablas del ejercicio anterior:

1. Analizar cómo debe definirse codigo_sector.
2. Modificar un empleado para quitarle temporalmente la asignación a un sector.
3. Intentar hacerlo utilizando una cadena vacía y luego utilizando NULL.
4. Identificar los empleados sin sector utilizando IS NULL.
5. Explicar por qué NULL no representa lo mismo que un código de sector inexistente.

Ejercicio 12. Eliminación de un sector

La empresa decide eliminar el sector Depósito.

Sin embargo, existen empleados que todavía tienen asignado ese sector.

Se requiere:

1. Intentar eliminar directamente el sector.
2. Analizar el resultado producido por la FOREIGN KEY.
3. Modificar primero los empleados afectados, asignándolos a otro sector o dejando su sector como NULL, según permita el modelo.
4. Eliminar posteriormente el sector Depósito.
5. Indicar por qué el **orden de las operaciones** es relevante.

Modificación de estructuras con datos

En los siguientes ejercicios el objetivo principal no es encontrar una única sentencia SQL, sino **reconocer el procedimiento de transformación de los datos y del modelo**.

Ejercicio 13. Separación de provincias cuando ya existe el código

Una base de datos posee la siguiente tabla:

1	Departamento
2	-----
3	codigo_departamento
4	nombre_departamento
5	codigo_provincia
6	nombre_provincia

Ejemplo de datos:

1	1 Capital	18 Corrientes
2	2 Goya	18 Corrientes
3	3 Oberá	54 Misiones
4	4 Eldorado	54 Misiones
5	5 Paraná	30 Entre Ríos
6	6 Concordia	30 Entre Ríos

Se observa que el nombre de la provincia se repite innecesariamente en numerosos departamentos.

Se requiere transformar el modelo para obtener:

1	Provincia
2	-----
3	codigo_provincia
4	nombre_provincia

y:

1	Departamento
2	-----
3	codigo_departamento
4	nombre_departamento
5	codigo_provincia

Se deberá:

1. Determinar el orden correcto de las operaciones.
2. Crear la tabla Provincia.
3. Poblirla recuperando **solamente combinaciones distintas** de código y nombre de provincia existentes en Departamento.
4. Establecer la relación correspondiente entre ambas tablas.
5. Eliminar de Departamento la columna que deja de ser necesaria.
6. Verificar que no se haya perdido información.
7. Explicar por qué **no sería conveniente eliminar** nombre_provincia **antes de poblar la nueva tabla**.

Ejercicio 14. Separación de provincias cuando no existe el código

En otro sistema se encuentra una situación diferente:

1	Departamento
2	-----
3	codigo_departamento
4	nombre_departamento
5	nombre_provincia

Datos:

1	1 Capital	Corrientes
2	2 Goya	Corrientes
3	3 Oberá	Misiones
4	4 Eldorado	Misiones
5	5 Paraná	Entre Ríos
6	6 Concordia	Entre Ríos

En este caso **no existe todavía un código de provincia**.

Se desea obtener:

1	Provincia
2	-----
3	codigo_provincia
4	nombre_provincia

dónde código provincia sea generado automáticamente, y modificar Departamento para conservar solamente dicho código como referencia.

Se requiere determinar y ejecutar los pasos necesarios para:

1. Crear Provincia.
2. Insertar en ella los distintos nombres de provincias existentes.
3. Generar automáticamente sus códigos.
4. Agregar codigo_provincia a Departamento.
5. Completar los códigos correspondientes en los departamentos existentes.
6. Se acepta realizar un UPDATE independiente para Corrientes, otro para Misiones y otro para Entre Ríos.
7. Establecer la FOREIGN KEY.
8. Eliminar finalmente nombre_provincia de Departamento.
9. Explicar por qué en este caso el procedimiento es diferente al utilizado en el ejercicio

Ejercicio 15. Normalización de categorías de productos con código existente

Una tabla de productos presenta la estructura:

1	Producto
2	-----
3	codigo_producto
4	descripcion
5	precio
6	codigo_categoria
7	nombre_categoria

Datos de ejemplo:

1	1 Teclado	25000	10 Periféricos
2	2 Mouse	18000	10 Periféricos
3	3 Monitor	210000	20 Monitores
4	4 Webcam	45000	10 Periféricos
5	5 Notebook	850000	30 Computadoras

Se desea crear una tabla independiente Categoria.

Se requiere:

1. Identificar qué datos deben pasar a la nueva tabla.
2. Crear Categoria.
3. Recuperar desde Producto los valores necesarios sin generar categorías repetidas.
4. Establecer la relación mediante FOREIGN KEY.
5. Eliminar la información redundante de Producto.
6. Indicar el orden de las operaciones.
7. Señalar en qué paso existiría riesgo de pérdida de información si las operaciones fueran realizadas en un orden incorrecto.

Ejercicio 16. Normalización de tipos de actividad sin código existente

Un pequeño sistema registra comercios de la siguiente manera:

1	Comercio
2	-----
3	codigo_comercio
4	nombre
5	tipo_actividad

Datos:

1	1 El Buen Pan	Panadería
2	2 La Espiga	Panadería
3	3 CentroTech	Informática
4	4 Norte Sistemas	Informática
5	5 San Martín	Farmacia

La empresa desea administrar los tipos de actividad en una tabla independiente:

1	TipoActividad
2	-----
3	codigo_tipo
4	descripcion

Actualmente no existen códigos para las actividades.

Se requiere:

1. Crear la nueva tabla utilizando un código IDENTITY.
2. Insertar los tipos de actividad sin duplicados.
3. Incorporar a Comercio una columna para almacenar el código correspondiente.
4. Actualizar los comercios existentes.
5. Crear la FOREIGN KEY.
6. Eliminar tipo_actividad de Comercio.
7. Enumerar el procedimiento de transformación antes de escribir las sentencias SQL.

Ejercicios de integración de procedimientos

Ejercicio 17. Datos existentes incompatibles con una nueva restricción

Una tabla ya contiene los siguientes productos:

1	codigo	descripcion	precio
2	1	Teclado	25000
3	2	Mouse	18000
4	3	Parlante	0
5	4	Monitor	210000
6	5	Adaptador	-1500

La empresa establece posteriormente la regla:

Todo producto debe tener un precio mayor que cero.

Se requiere:

1. Crear mediante ALTER TABLE una restricción CHECK apropiada.
2. Analizar qué problema presentan los datos actuales antes de incorporarla.
3. Identificar los registros incompatibles.
4. Evaluar que procedimientos utilizar (Actualizar o no los datos?).
5. Incorporar finalmente la restricción.
6. Intentar insertar un nuevo producto con precio negativo.
7. Explicar por qué, en una base que ya contiene datos, puede ser necesario **corregir primero los datos y modificar después la estructura.**

Ejercicio 18. Incorporación progresiva de una relación

Una aplicación registra reservas:

1	Reserva
2	-----
3	codigo_reserva
4	fecha
5	nombre_sala

Los valores actualmente utilizados para nombre_sala son:

1	Auditorio
2	Laboratorio 1
3	Laboratorio 2

Como estos nombres se repiten en muchas reservas, se decide crear:

1	Sala
2	----
3	codigo_sala
4	nombre_sala

Cada sala deberá poseer un código generado automáticamente y un nombre único.

La tabla Reserva deberá terminar conteniendo *codigo_sala* en lugar de *nombre_sala*.

Se requiere **sin eliminar las reservas existentes**:

1. Analizar la estructura de partida.
2. Proponer el orden de transformación.
3. Crear Sala.
4. Cargar los nombres distintos existentes.
5. Incorporar la nueva columna a Reserva.
6. Completar sus valores mediante UPDATE. Se permite realizar una sentencia por cada sala.
7. Verificar que todas las reservas tengan su código correspondiente.
8. Crear la FOREIGN KEY.
9. Eliminar *nombre_sala* de Reserva.
10. Explicar qué podría ocurrir si se intentara:
 - eliminar primero *nombre_sala*;
 - definir *codigo_sala* como obligatorio antes de completar sus valores;
 - crear la FOREIGN KEY cuando existen códigos que no pertenecen a ninguna sala.